

Open content

Exact direct-use workflows and research diagnostics

Bounded integer design, globally cheapest experiments, algebraic graph sampling, nonlinear sequence execution, distinct populations, symbolic joins, compatible charts and a local object console.

8

persistent object kinds

52

nonlinear families

512

family-partitioned tasks

Full source and evidence without an archive cap. Tested Python and measured C kernels; general mathematical proofs, global smooth geometry and physical derivations retain their stated open scope.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

What this release supplies

This release completes concrete software extensions from the direct-use roadmap and retains the research content in full. The output is an executable system of eight persistent mathematical object kinds, exact original-coordinate queries, portable experiments and local clients. There is no archive size cap. Existing Lean sources, prior evidence, source snapshots and historical research notes remain in the repository.

The direct beneficiaries are engineers who need coupled configurations, calibration authors who need integer settings under inequalities, benchmark builders who need entire held-out mathematical families, recurrence authors who need huge-index execution, gain-network researchers who need exact algebraic random decisions, geometry researchers who need compatible local coordinates, and physics researchers who need a precise test of a model's identifiability. These are distinct workflows with stated contracts; no universal return or market valuation follows from their existence.

The dense comparison in `DIRECT_USE_BUILD_ROADMAP.md` gives every application, its conventional counterpart, its new executable output and the remaining scope. The current receipts are under `receipts/open_content`. The previous application receipts remain historical: their conventional Python matrix baseline won. Current graph sampling uses a newer receipt schema, and global experiment selection supersedes witness-only selection, so older transcripts are historical outputs rather than byte-identical expectations for changed APIs.

Distinct populations and symbolic joins

ExactPopulation already supplies a reusable address space for a finite supported integer domain or curve. A rank identifies an original source parameter or disjoint original-coordinate point. Derived field values may collide. Sampling source ranks uniformly therefore need not sample distinct projected values uniformly.

ProjectedPopulation supplies a separate identity contract for constant, linear and quadratic domain fields. For $f(n)=a n^2+b n+c$, the difference $f(n)-f(m)$ factors as $(n-m)(a(n+m)+b)$. If a is nonzero, a distinct integer collision must be the reflection $m=-b/a-n$. A nonintegral reflection makes the integer map injective. Otherwise the owner predicate retains n when it is the smaller source parameter, or when its reflection is outside the source domain. Source constraints, including Boolean and modular predicates, are substituted exactly. The resulting representative domain has one owner per distinct image value.

Counts, ranks, samples and shards then reuse the existing complete domain engine. Locate solves the integer image equation and selects the least valid original parameter; multiplicity counts valid preimages. A constant field has one image value when its source is nonempty, with multiplicity equal to source cardinality. An empty source has no image values. Value ranks follow increasing canonical owner, not increasing numeric field value.

The recorded source n in $[-10^{50}, 10^{50}]$ has $2 \times 10^{50} + 1$ identities and $10^{50} + 1$ distinct square values. Zero has multiplicity one; every positive square in the image has

multiplicity two. The example samples sixteen distinct values without enumerating the enormous source. Forty-five independently exhausted asymmetric/modular quadratic cases test ownership and multiplicities against literal sets.

Symbolic equality joins compile $f(x)=g(y)$ together with both original source predicates as a supported complete curve population. Original parameter pairs are the identities. The square self-join over the same interval contains $4 \times 10^{50} + 1$ pairs: two signed pairs for each nonzero source parameter and one zero pair. Complete curve charts supply count, rank, locate and sampling without a materialized join table. Unsupported curve relations fail; this is not a general symbolic relational database or a solver for a two-dimensional constant plane.

Bounded integer calibration with inequalities

`InverseDesign.solve_box` adds finite integer bounds and up to 128 linear equality or inequality constraints to exact observation equations. The target and positive definite metric may be rational. The result contains every minimizing integer setting inside the supplied box, its exact energy, the full model and replay information. An empty feasible set is reported explicitly.

The search bounds each residual linear constraint over the unfixed coordinate box. A prefix is impossible if the attainable interval cannot meet the required equality or inequality. For the objective, eliminating the remaining real coordinates gives an exact Schur-complement lower bound. An energy strictly larger than the incumbent is pruned; equality is retained so all optimal ties survive. Candidate values are generated lazily in order of distance from their target coordinate. A node-budget failure returns no completed optimum.

The application example has three nonnegative integer actuator increments, two exact observations, individual upper limits, a relative-setting inequality and a coupled-capacity inequality. An independent eliminated-variable scan verifies its exact minimum. Twenty-five randomized three-variable models with rational targets and off-diagonal metrics are compared against every point in $[-3,3]^3$. These checks substantiate the supplied finite-box algorithm. They do not fit physical sensor data, implement nonlinear tolerances, or establish an optimum outside the box.

Globally cheapest separating experiments

A certificate's readout words prove that two machine states can be distinguished. They need not be the cheapest way to do it. The new search treats each exact compressed observable difference as a state in a weighted directed graph. Applying an operator gives an edge with strictly positive supplied cost. Reading an output adds a nonnegative supplied readout cost.

A known certificate witness initializes a finite upper bound. Dijkstra search explores only paths that can improve it; reaching the same exact difference more expensively is dominated. Every readout is tested at each visited difference. A queue whose smallest

possible total cost reaches the incumbent cannot improve it. Positive edge costs and the finite incumbent bound the relevant word lengths. Within the state and integer-bit budgets, the resulting cost is a global minimum over all finite chronological operator words and readouts, not merely the stored witness set. One minimizing experiment is returned.

The supplied two-state example has an immediate expensive readout costing 100. One transition exposes the difference to a cheap readout, for total cost two. The historical witness-only method still exposes the comparison. A second test uses two operators with different prices and compares the answer against independently evaluated short words. States with identical observable futures return `ALL_FUTURE_EQUAL`. Adaptive noisy experiments and zero-cost operator cycles are outside this contract.

Exact algebraic graph sampling

GraphEnsemble now samples the specified gain-graph basis measure for every supported cyclotomic order from two through 64. Include/exclude conditioning and immutable weight repairs retain their previous contracts. Rational conditional probabilities use exact integer draws. Nonrational real probabilities use a lazy uniform binary real and rational enclosing intervals for the probability.

The real embedding is $z = \exp(2\pi i / \text{order})$. Machin's identity supplies rational enclosures for π through alternating arctangent series. Cosine Taylor polynomials have explicit remainder bounds, and interval angle uncertainty is enclosed with the cosine Lipschitz bound. Rational outward rounding keeps the resulting endpoints compact. Exact conjugation first checks that the field element is real. Linear combinations of cosine enclosures give an interval containing the chosen algebraic probability.

Each random bit narrows the uniform interval. Inclusion is decided only when its entire interval lies below the probability interval; exclusion only when it lies above. Precision refines as necessary. A random-bit or arithmetic work failure returns no sample. Every accepted decision stores the exact field coefficients, precision, random prefix, both intervals and inclusion result. `verify_decision` recomputes the bounds and separation inequality; it verifies the decision against its recorded prefix, not the statistical quality of a user's RNG.

The release records 504 samples across all 63 supported orders. Every stored decision is replayed. An independent order-five example checks the known quadratic probability using integer square-root bounds for $\sqrt{5}$, and tampering with an inclusion result is rejected. The measure remains the declared gain-graph basis model; no unsigned-network interpretation or distribution-speed advantage is inferred. The analytic enclosure implementation is exact Python with mathematical analytic premises, not a Lean-verified sampler.

Machin identity reference: <https://www.math.brown.edu/~res/M10/machin.pdf> . Related verified lazy-sampling work: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.LICS.2026.71> . That separate

Rocq verification does not certify this implementation.

Reviewed linear and nonlinear sequence execution

Four local source snapshots are reviewed against explicit recurrences, initial states and offsets: Fibonacci A000045, Lucas beginning at two A000032, Lucas beginning at one A000204, and Pell A000129. Fibonacci and both Lucas views share a two-state Fibonacci transition; Pell has its own two-state transition. Every available stored term is independently checked. Source file hashes and offsets accompany the definitions. This accepts the definitions as premises; matching a prefix does not infer an all-future law for an unknown sequence.

FactorialLibrary adds a separate nonlinear class: supplied products and quotients of $(c\ n)!$ with positive integer slopes. Each family has an exact polynomial-coefficient hypergeometric recurrence. Balanced families have a separate Landau integrality certificate. Common stripped factorial units at the same slope are computed once per index/prime/depth and reused by every readout.

At enormous indices, factorial-unit recursion strips prime factors before modular inversion. The difference of factorial valuations determines p-adic integrality. A nonnegative valuation yields a residue modulo p^{depth} , including zero when the valuation reaches the depth. A negative valuation returns NON_P_INTEGRAL with no residue. Singular recurrence denominators therefore do not force an invalid modular inverse.

All 52 parameter families from the existing sourced Bober table are integrated. Twelve small terms per family give 624 checks against independently computed literal factorial ratios. Full receipts at index 10^{100} modulo 7^3 and 17^2 retain shared units and recursive levels. This enables direct huge-index modular experiments without stepping through a recurrence. It does not solve arbitrary nonlinear recurrences or prove a general Gamma perfect-power classification.

Bober parameter source, already preserved with a source hash:
<https://arxiv.org/html/0709.1977v1> .

Whole-family mathematical tasks

The new corpus splits entire declared exponent families. Four families are training sources, two are validation sources, and two are test sources. Each contributes 64 tasks, for 256 training, 128 validation and 128 test tasks. Signed siblings stay inside a source family. A declared family or canonical population alias cannot cross partitions, and duplicate task identities are rejected.

Public JSONL files contain the source definition, operation and original values, with no private rank answer. `solve_public_tasks` reconstructs each population from those prompt fields, reuses compiled definitions and executes `locate`. Its recorded 128/128 test result is measured exact-solver performance on public prompts, with elapsed query times. It is not oracle replay or LLM evaluation. The older corpus's oracle replay remains labeled as such.

A benchmark evaluator should receive only public test tasks until predictions are frozen. `heldout_private.json` remains in this full research archive for reproduction and should be withheld during blind evaluation. Family names and canonical definitions cannot rule out every differently encoded mathematical equivalence; meaningful semantic family separation remains a source-review responsibility. These are constructed arithmetic tasks, not a broad claim of mathematical reasoning generalization.

Compatible local chart geometry

The workbench now carries certified branch-to-finite and infinity-to-finite overlaps. A branch map is $x=b+t^2$. Factoring $f(b+t^2)=t^2 q(b+t^2)$ identifies the curve equation, while the density Jacobian is $4|t|^2$. Infinity uses $x=t^{-2}$ for odd polynomial degree and $x=t^{-1}$ for even degree. The reciprocal polynomial identity and exact radial exponents account for the metric pullback.

Bernstein certificates prove that the whole supplied source rectangle lies inside the target finite rectangle, denominators are positive and t is nonzero. `verify_transition` reconstructs the maps and identities and checks the witness polynomials, signs and boxes. An unresolved certificate does not authorize transport. Exact rational points are transported through the map; target density bounds multiplied by the exact Jacobian intersect source density bounds. The interactive client displays these transports alongside the original point and local path interval.

The example uses $y^2=x^3-x$, with finite x in $[2,3]$, a branch rectangle near $t=1.55$ and an infinity rectangle near $t=0.625$. Both entire rectangles have certified images in the finite panel. Odd and even infinity formulas are tested through polynomial degree nine, genus four. The Bernstein per-axis degree ceiling is raised to 64 because the already supported degree-nine metric needs degree 36; other work and monomial budgets remain explicit.

This supplies compatible local patches. It does not establish a full atlas covering the compact curve, certify normalized numerical period integration, identify smooth metrics with a mesh, solve global geodesics or produce arbitrary-genus smooth Voronoi boundaries. Numerical display colours are approximate; the readouts and transports retain exact rational enclosures.

Persistent objects and the local client

The SQLite catalogue now supports population, projected, sequence, factorial, inverse, graph, geometry and combinatorial kinds. Content addresses identify canonical supplied definitions. Aliases and compile reuse preserve previous behavior. A symbolic join registers a derived curve population. Geometry transitions, bounded calibration, global experiments and factorial residues are explicit service methods; source evaluation is never a request operation.

The HTTP console binds only to 127.0.0.1 and uses a single worker. It serves an interactive request editor and the existing bounded JSON protocol. Host/origin checks, bounded bodies, method allowlists and isolated errors are exercised through actual HTTP

round trips. SQLite access may move from the creating thread to that one serving worker; this is not concurrent shared-connection execution.

The browser sends the original JSON request text, and displays the raw response text so large integer digits are preserved. Parsing is used for request validation, not for converting exact results into JavaScript numbers. The offline geometry view uses approximate midpoints only for colours. Native browser screenshot verification is unavailable on this host; the release uses executable DOM/canvas script checks and actual HTTP tests. Public deployment, user authentication and concurrent hosting are separate work.

Real compiled configuration measurements

The source includes conventional C99 dot products, a compatible tiled C kernel and a cache-friendly untiled C control. All are compiled together with identical optimization flags. Inputs and matrix size are bounded so every uint64 result is exact. An independent Python dot-product computation supplies the full expected output, and every warmup and measured output is checked byte for byte.

Eight compatible tile parameters fit the 262,144-byte logical full rectangular output-tile budget. Rows are n^3 , columns n^2 and the logical footprint is $8n^5$ bytes. Boundary tiles are clipped to the matrix. The budget describes a logical output footprint, not an allocated scratch array or measured cache residency.

Seven measured rounds follow two warmups at each matrix order, 96 and 192. Candidates and both controls are interleaved in seeded shuffled order. Every trial, compiler version, flags, source hash, CPU and output hash is preserved. The final numerical results are in receipts/open_content/summary.json and compiled_tuning.json. Compatible configurations beat the conventional dot-product kernel on these examples; the cache-friendly untiled control is faster still. The loop organization explains much of the gain over dot products. A perfect-power dimension relationship is not itself a cache optimization theorem.

The direct output is a measured best compatible configuration for a specified workload. Conventional unconstrained tuning or an optimized BLAS implementation may provide different or faster choices. These are two small bounded integer products on one host, not a universal hardware result. The earlier conventional Python win remains in its original receipts.

Flavor identifiability and interaction obligations

The scalar model declares $x=2\cos \theta$ and rational CP-even harmonic coefficients through degree twelve. There are 13 coefficients, including the constant. Preserving every root of the real cyclotomic carrier Ψ_{60} requires the potential derivative to vanish modulo that degree-eight carrier. Reducing each basis derivative gives an exact rational eight-by-13 constraint matrix.

Its rank is eight and its kernel has dimension five. Each kernel vector is multiplied through the matrix and checked exactly. Thus stationarity imposes eight independent linear conditions within this finite declared coefficient space. The constant is one free direction. This establishes stationarity, not global minimality, a unique selected vacuum or a UV mechanism enforcing those conditions.

The release also computes the first-order response to the permitted CP-even deformation $V_\varepsilon(x) = V_0(x) + \varepsilon x$ at a nominated primitive embedding. Exact implicit differentiation gives $dx/d\varepsilon = -1/V_0''(x)$. The scalar $C = 1 - z^{12} - z^{-12}$ initially obeys $C^2 - 3C + 1 = 0$ and lies between 0.38 and 0.39 at the chosen embedding. Its first-order response changes that golden polynomial: $(2C - 3)dC/d\varepsilon$ is nonzero. The reference stationarity, golden relation and curvature response identity are checked in the exact cyclotomic field, with certified real embedding intervals. Two lock choices are recorded.

This is a concrete permitted perturbation showing that the nominated scalar value and phase are not protected by the declared symmetries alone. It neither identifies C with a physical CKM coefficient nor derives physical mixing. The complete allowed Yukawa operator audit is retained. Required unresolved inputs are a noncentral charge-compatible interaction and field representations, a three-light-family mechanism, correlated up/down breaking, canonical normalization/matching, and protected vacuum selection under renormalization. A numerical nomination of φ^{-2} or 66 degrees cannot replace those inputs.

Full open content and proof boundaries

The inventory retains the full text and hashes of OPEN_FRONTS.md, OPEN_PROBLEMS.md, FRONTIER_PLAN.md and M22_TRANSPORT_INTERACTIONS.md, with source lines for paragraphs mentioning open work, hypotheses or obstructions. These notes span multiple releases. Some earlier gaps have narrower later closures; a keyword match is therefore SOURCE_NOTE_REQUIRES_CURRENT_PROOF_REVIEW, not a claim that every sentence describes an unresolved theorem today. The updated application roadmap gives the current software contracts directly.

This release adds no Lean theorem and does not certify its Python producers in the kernel. Lean 4.20.0 is installed, but the pinned Mathlib artifacts are unavailable here; the full Lean build and axiom audit were not rerun. Existing committed kernel evidence remains preserved, with its previous scope. Four optional Lean-dependent Python checks remain skipped in this environment.

General Sturm transcript correctness, saturation and enumeration formalization, broader effective Baker/number-ring pipelines, remaining Matveev premises, larger descent slab work, analytic period normalization, smooth global geometry and the UV flavor interaction remain open. Hall, Pillai, abc and uniform integral-point bounds are not proved by finite examples or this release. No historical conjecture is marked closed by a new interface.

Reproduction and review

Run from a repository checkout. Python 3.10 or later supplies the arithmetic and service runtime; a C99 compiler supplies the optional compiled benchmark. Document rendering uses optional reportlab/matplotlib. Timings vary by host; all mathematical outputs and fixed-seed decisions are reproducible.

```
export PYTHONPATH=python
python -m unittest discover -s python/tests -p test_open_content.py
python python/develop_open_content.py --output /tmp/pp-open
python -m perfectpower service --database /tmp/pp-open.sqlite \
    < receipts/open_content/service_requests.jsonl
python -m perfectpower service --database /tmp/pp-open.sqlite --http-port 8080
make test
```

Open the recorded `geometry_workbench.html` locally to inspect compatible panels. Current receipts include all eight object definitions, request/response replay, exact graph decisions, bounded design, global experiment, reviewed linear sources, all nonlinear family evidence, huge distinct-value and join examples, private/public family tasks, measured baseline results, compiled trial ledger, full flavor audit and complete historical source inventory. The three largest full receipts use deterministic lossless `.json.gz` storage; `python python/unpack_open_content.py` creates plain JSON copies. No record is omitted. The accompanying source archive preserves the complete committed tree without a 30 MB cap.